Soluciones para el CLENA

CLENA-Catálogo Biblioteca

Arquitectura del proyecto

Presentado por:

Joshua Lobo

Kesly Rodriguez

Santiago Vengoechea

Luis Robles

Docente:

Daniel Romero



Tabla de contenido

1. Introducción *3*

2. Objetivos y alcance *3*

3. Visión general del sistema *3*

4. Modelo C4 ..... *4*

4.1. Nivel 1: Contexto del sistema **4**

4.2. Nivel 2: Contenedores **5**

4.3. Nivel 3: Componentes de la API **6**

5. Organización mediante Clean Architecture *6*

6. Estructura de carpetas del backend *7*

7. Inyección de dependencias en NestJS *8*

8. Decisiones técnicas *9*

8.1. ORM y persistencia: Prisma **9**

8.2. Validación: class-validator y class-transformer **9**

8.3. Logger: Pino **10**

8.4. Componentes del frontend **10**

9. Estrategia de búsqueda tolerante a errores *10*

10. Conclusiones *11*

11. Referencias *11*

1. Introducción

La Corporación Luis Eduardo Nieto Arteta (CLENA) es una entidad cultural de Barranquilla que administra, entre otros espacios, la Biblioteca Piloto del Caribe y el Archivo Histórico del Atlántico [1]. En este contexto se desarrolla Consulta Fácil, una aplicación web orientada a facilitar la consulta del catálogo y la localización de material bibliográfico. CLENA es la organización destinataria; Consulta Fácil es el sistema de software que se documenta en este informe.

La solución organiza registros de libros, autores, temas y ejemplares, además de gestionar cuentas y permisos. El repositorio incluye un módulo de préstamos, pero su presencia técnica no debe confundirse con el alcance mínimo evaluable en esta entrega: el enunciado permite completar los préstamos y las alertas de devolución en sprints posteriores [2]. La arquitectura separa las reglas bibliotecarias de las capas de presentación y persistencia para reducir el acoplamiento entre tecnologías y lógica de negocio.

Se documenta la arquitectura mediante los tres primeros niveles del modelo C4 de Simon Brown: contexto, contenedores y componentes [3]. Estas vistas se complementan con la organización por capas de Clean Architecture [4], el árbol de archivos del backend, la inyección de dependencias y las decisiones tecnológicas. El alcance descrito toma como referencia la implementación consultada en la rama dev del repositorio del equipo, sin equiparar esa rama con la versión definitiva de entrega.

2. Objetivos y alcance

El objetivo principal es explicar la distribución de responsabilidades, los límites del sistema y sus comunicaciones. Cada vista se concentra en un nivel de abstracción diferente y permite relacionar las funcionalidades bibliotecarias con los elementos que las soportan. Las capacidades previstas se distinguen de los componentes presentes en el código cuando afectan la interpretación del diseño.

- Delimitar los actores y el sistema desde la perspectiva del CLENA (C4 nivel 1).
- Identificar la aplicación React, la API NestJS y PostgreSQL, con sus principales comunicaciones (C4 nivel 2).
- Explicar el flujo de búsqueda del catálogo y sus dependencias dentro de la API (C4 nivel 3).
- Justificar las tecnologías de persistencia, validación, registro de eventos y presentación, y documentar la composición de dependencias en NestJS.

La vista de componentes se concentra en el flujo de búsqueda bibliográfica y no pretende listar todas las clases de la API. El nivel 4 de C4 es opcional para esta entrega: el detalle de la inyección se expone mediante un fragmento del módulo NestJS, sin crear una cuarta figura [2, 3].

3. Visión general del sistema

Consulta Fácil comprende una interfaz React/TypeScript y una API NestJS/TypeScript. La aplicación web consume recursos HTTP del backend bajo el prefijo /api/v1; Prisma Client conecta la capa de infraestructura con PostgreSQL 16. La estructura del backend incluye módulos de autenticación, catálogo y préstamos. Docker Compose levanta en local el frontend, la API y la base de datos; Swagger documenta los endpoints en /docs [5].

- Interfaz: React, TypeScript y Vite; en el despliegue Docker local, el frontend se publica en el puerto 5173 y sirve la aplicación web.
- API: NestJS y TypeScript para controladores, casos de uso, validación HTTP y exposición de recursos.
- Persistencia: Prisma Client 6 y PostgreSQL 16 para entidades bibliográficas y cuentas de usuario.
- Ejecución local: Docker Compose coordina el frontend, la API y la base de datos.

4. Modelo C4

El modelo C4 establece vistas jerárquicas para comunicar la arquitectura a distintos niveles: el contexto identifica actores y sistemas, los contenedores muestran unidades ejecutables y los componentes explican responsabilidades dentro de uno de esos contenedores. Las convenciones empleadas siguen las recomendaciones oficiales de nomenclatura, leyendas y relaciones etiquetadas [3]. Los tres diagramas son elaboraciones del equipo en Draw.io.

4.1. Nivel 1: Contexto del sistema

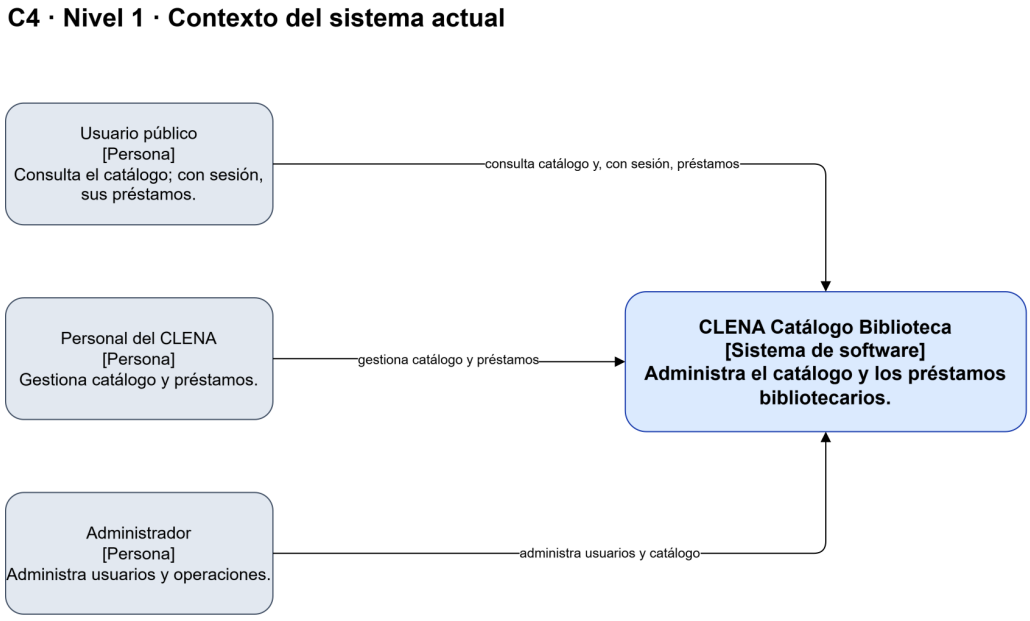


Figura 1. C4 nivel 1: contexto del sistema actual

Gris: persona o rol externo. Azul: sistema de software dentro del alcance. Los tres roles utilizan las vistas desde un navegador; en el entorno local la conexión emplea HTTP.

El sistema Consulta Fácil aparece como una única unidad. El usuario público consulta registros bibliográficos y, cuando dispone de sesión y de la funcionalidad habilitada, información relacionada con sus préstamos. El personal del CLENA administra registros del catálogo; el administrador gestiona usuarios y permisos. El diagrama registra los tres actores del enunciado sin introducir detalles tecnológicos internos que corresponden al nivel 2.

Las interacciones utilizan el navegador y HTTP en el despliegue local. No aparecen configurados servicios externos de correo ni de almacenamiento. La importación CSV recibe un archivo y lo procesa internamente: no constituye una integración con otro sistema.

4.2. Nivel 2: Contenedores

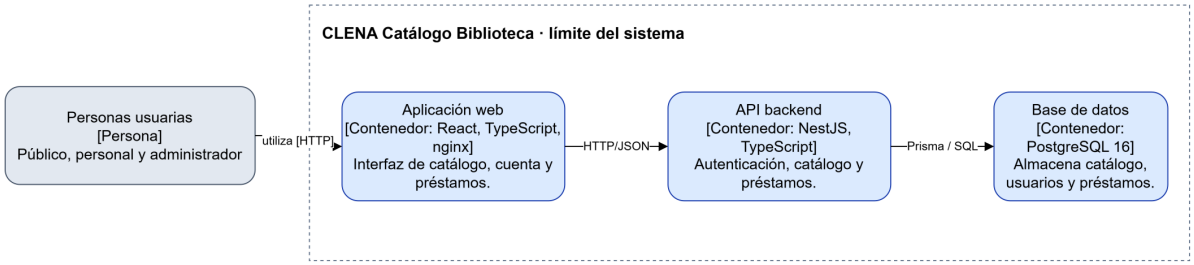


Figura 2. C4 nivel 2: contenedores y conexiones del sistema.

Gris: personas usuarias. Azul: contenedores del sistema. Borde punteado: límite de Consulta Fácil. HTTP/JSON para frontend–API y protocolo PostgreSQL (mediante Prisma) para API–BD.

La aplicación React sirve la interfaz y envía al backend solicitudes del catálogo y de autenticación. La API NestJS coordina los casos de uso y consultas a la base de datos. PostgreSQL 16 conserva los registros persistentes; la aplicación se sirve por HTTP en Docker Compose y las comunicaciones con la API usan JSON [5]. El uso de HTTPS depende de una configuración de despliegue que no se representa aquí.

Relación	Contenido intercambiado	Protocolo / tecnología
Navegador → React	Vistas, recursos estáticos y acciones de usuario.	HTTP (entorno local).
React → NestJS	Solicitudes y respuestas de catálogo, sesión y cuenta.	HTTP/JSON, API REST /api/v1.
NestJS → PostgreSQL	Consultas y escrituras bibliográficas y de usuarios.	Prisma Client; SQL sobre protocolo PostgreSQL.

El CSV se recibe mediante la API de importación y es procesado dentro del backend; no requiere representar un cuarto contenedor externo. Tampoco se identifican, dentro de este despliegue, un servidor SMTP ni un servicio independiente de almacenamiento de archivos.

4.3. Nivel 3: Componentes de la API

Esta vista amplía el contenedor API backend y se concentra en el flujo de búsqueda bibliográfica. Las cajas se agrupan en presentación, aplicación, dominio e infraestructura; se distinguen los componentes concretos de los contratos que utilizan.

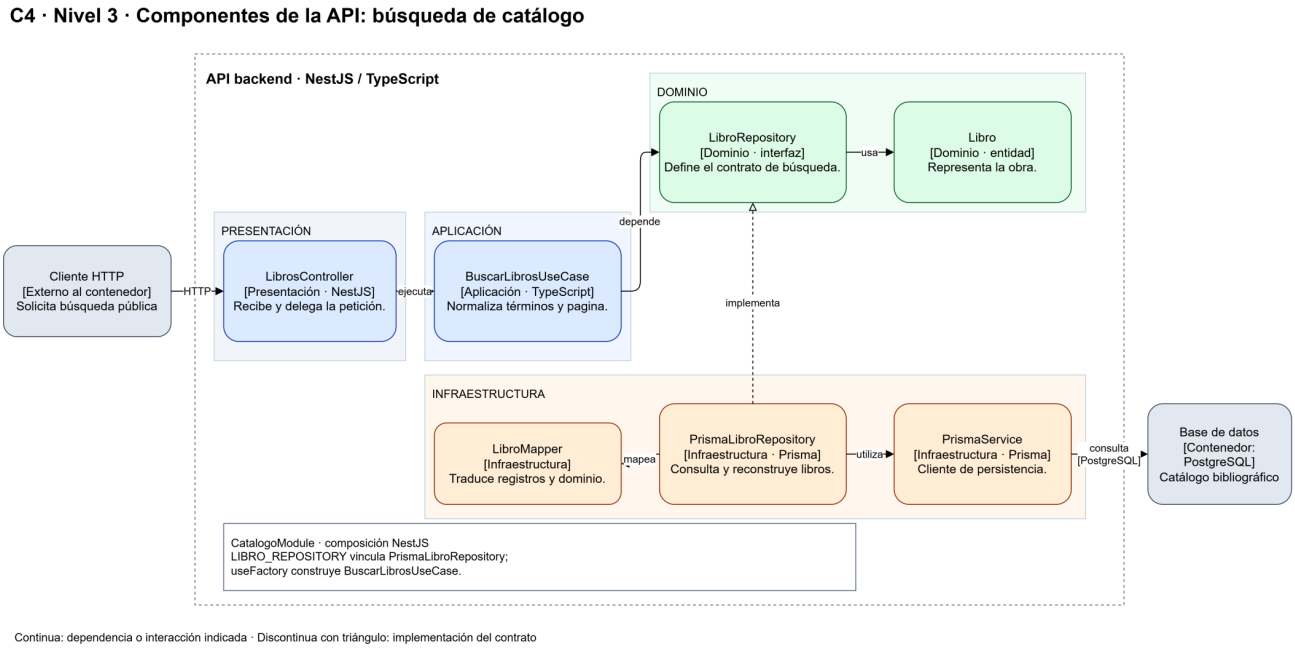


Figura 3. C4 nivel 3: componentes de la API para búsqueda de catálogo

5. Organización mediante Clean Architecture

La organización aplica la regla de dependencia de Clean Architecture: las políticas y entidades del dominio no necesitan conocer herramientas de presentación ni persistencia [4]. En el módulo de catálogo, BuscarLibrosUseCase depende de la interfaz LibroRepository y no de PrismaLibroRepository. Esta distribución deja el contrato en una capa interna, mientras la implementación Prisma permanece en infraestructura.

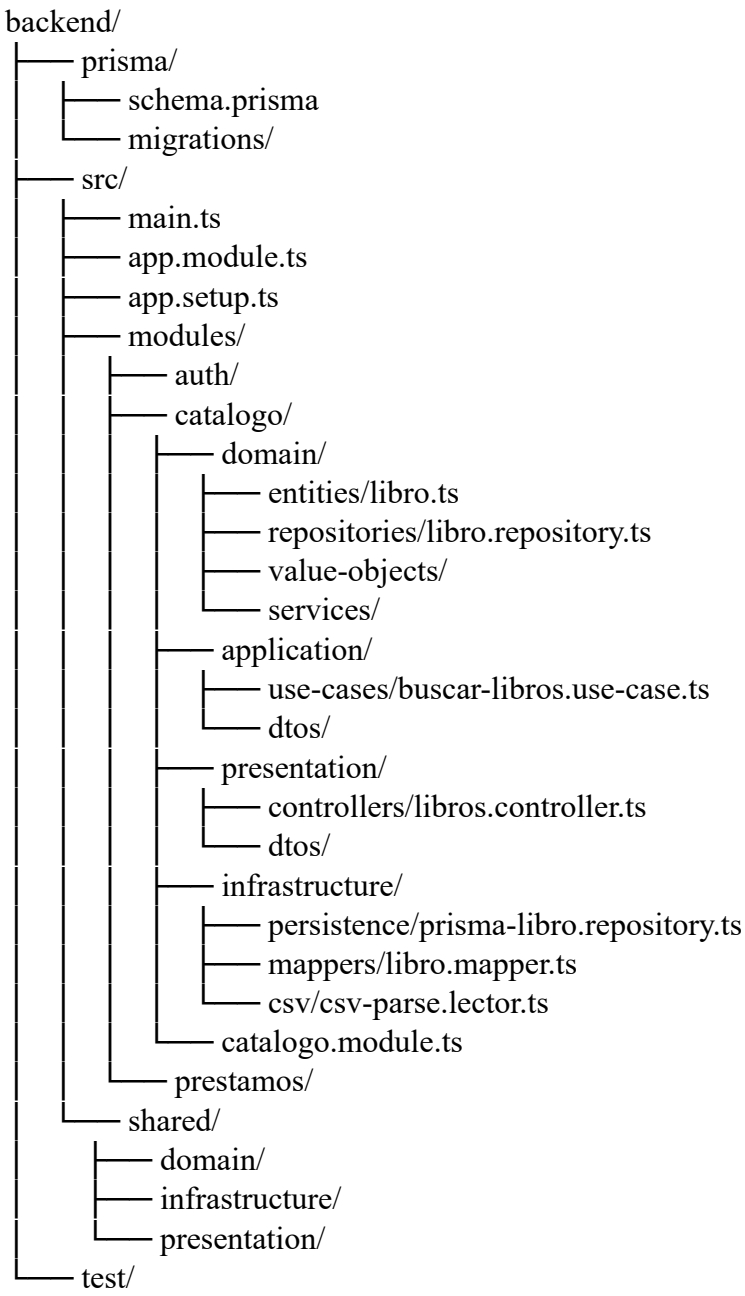
Capa	Responsabilidad	Ejemplos del proyecto
Dominio	Entidades, objetos de valor, reglas de negocio y contratos.	Libro, Ejemplar, Isbn, LibroRepository.
Aplicación	Coordinación de acciones y DTOs de entrada/salida de los casos de uso.	BuscarLibrosUseCase.

Presentación	Exposición HTTP, validación de solicitudes y entrega de respuestas.	LibrosController, DTOs de petición, filtro de excepciones.
Infraestructura	Adaptación a persistencia y herramientas externas.	PrismaLibroRepository, LibroMapper, PrismaService, logger.

La separación es verificable en los imports de las clases principales: LibroRepository referencia tipos del dominio; BuscarLibrosUseCase usa el contrato, y el módulo NestJS asocia ese contrato a PrismaLibroRepository. El dominio de catálogo analizado no requiere importar NestJS ni Prisma para declarar su entidad Libro o el contrato de búsqueda. Presentación e infraestructura son capas externas respecto de las políticas internas: su posición en la figura no establece una dependencia obligatoria entre ambas [4, 5].

6. Estructura de carpetas del backend

El backend se organiza por módulos funcionales. Dentro de catalogo/, los directorios separan responsabilidades arquitectónicas. La siguiente vista incluye ejemplos concretos del módulo de búsqueda y omite archivos que no aportan a esta explicación:



Los modelos de negocio y puertos de repositorio se localizan en domain; application contiene los casos de uso; presentation agrupa la entrada HTTP; e infrastructure implementa repositorios, mappers y lectores CSV. CatalogoModule compone las dependencias de NestJS. El código transversal reside en shared y el esquema físico y las migraciones se mantienen bajo prisma/. Esta distribución permite encontrar el responsable de cada tipo de operación sin mezclar clases de dominio con adaptadores de base de datos.

7. Inyección de dependencias en NestJS

CatalogoModule registra el token LIBRO_REPOSITORY y vincula su implementación a PrismaLibroRepository. La fábrica del caso de uso recibe el contrato LibroRepository a través del arreglo inject, conforme al mecanismo de custom providers de NestJS [5]. El fragmento se ha tomado de backend/src/modules/catalogo/catalogo.module.ts, rama dev; se han omitido otros providers del mismo arreglo para concentrar la explicación:

```
providers: [
        { provide: LIBRO_REPOSITORY, useClass: PrismaLibroRepository },
        // Otros providers del módulo omitidos.
        {
        provide: BuscarLibrosUseCase,
        useFactory: (libros: LibroRepository) => new BuscarLibrosUseCase(libros),
        inject: [LIBRO_REPOSITORY],
        },
]
```

El token existe en domain/repositories/libro.repository.ts. Como las interfaces TypeScript desaparecen en tiempo de ejecución, la asociación explícita permite que NestJS entregue una implementación concreta al construir el caso de uso. Esta técnica favorece las pruebas con dobles de repositorio y evita que la lógica de aplicación deba importar la implementación basada en Prisma.

8. Decisiones técnicas

8.1. ORM y persistencia: Prisma

El backend utiliza Prisma Client 6 para consultar PostgreSQL mediante repositorios de infraestructura, con modelos tipados y migraciones versionadas [6]. Para un catálogo con relaciones entre libros, autores, temas y ejemplares, los tipos generados reducen errores de acceso y facilitan consultas sobre entidades relacionadas. La conversión mediante LibroMapper preserva la distinción entre modelo persistido y entidad del dominio.

Frente a TypeORM, que puede modelar entidades mediante decoradores y ofrecer repositorios asociados a esas clases [7], Prisma concentra las definiciones relacionales en schema.prisma y genera un cliente tipado. Esta separación resulta adecuada cuando se busca evitar que las entidades del dominio sean también entidades del ORM. Como contrapartida, el equipo mantiene mappers y SQL específico para índices y búsqueda por trigramas, que Prisma no sustituye.

8.2. Validación: class-validator y class-transformer

Los DTOs HTTP utilizan class-validator y class-transformer, integrados con ValidationPipe de NestJS. El pipe transforma parámetros, valida restricciones declaradas y rechaza propiedades no permitidas, mientras el dominio comprueba invariantes propios del catálogo [8]. Así se distingue entre validez del formato de una petición y validez de una operación de negocio.

Una alternativa basada en esquemas, como Zod, permite centralizar validaciones sin decoradores. En este proyecto se mantiene el enfoque de DTOs de clase por su integración directa con NestJS y las descripciones de Swagger. Su límite es que no reemplaza la validación de reglas de negocio en los casos de uso y las entidades.

8.3. Logger: Pino

El backend integra nestjs-pino y pino-http para producir registros estructurados en JSON, con identificadores de solicitud y niveles configurables. La configuración del proyecto oculta determinadas cabeceras sensibles, lo que ayuda a diagnosticar fallos HTTP sin incorporar lógica de observabilidad a las entidades [9].

Winston constituye una alternativa flexible cuando se requieren numerosos transportes de salida y formatos de registro [10]. Pino se adapta al entorno actual porque proporciona registros JSON de bajo costo operativo y una integración sencilla con el ciclo de petición de NestJS. Esta elección no implica por sí sola que exista almacenamiento centralizado ni que todos los datos personales queden automáticamente anonimizados.

8.4. Componentes del frontend

El frontend combina React con componentes propios y estilos CSS Modules. No incorpora una biblioteca general de componentes como Material UI; lucide-react cumple una función de iconografía y @fontsource/outfit aporta la tipografía, como refleja el package.json del frontend [11]. La elección de componentes propios permite adaptar formularios, filtros y vistas al lenguaje visual del catálogo y controlar el comportamiento de accesibilidad desde su implementación.

Material UI ofrecería componentes preconstruidos y una base de diseño consistente [12], lo que podría acelerar interfaces extensas. Mantener componentes propios evita añadir un sistema visual externo para las pantallas actuales, aunque exige al equipo probar por cuenta propia el contraste, la navegación con teclado, las etiquetas de formulario y la consistencia visual. Esta es una justificación técnica de la solución existente, no una afirmación sobre una comparación histórica documentada.

9. Estrategia de búsqueda tolerante a errores

La búsqueda bibliográfica pública se expone mediante GET /api/v1/libros/buscar. La entrada incluye un texto q y parámetros de paginación; el caso de uso normaliza espacios y términos y delega la consulta al contrato LibroRepository. La implementación de infraestructura combina coincidencias directas y aproximadas en título, autor y tema.

PostgreSQL incorpora pg_trgm para evaluar similitud mediante trigramas y unaccent para eliminar marcas diacríticas del texto [13, 14]. La migración versionada del repositorio habilita ambas extensiones, crea la función f_unaccent y define índices GIN sobre expresiones normalizadas de títulos, autores y temas. El repositorio utiliza estas capacidades para combinar coincidencias literales y aproximadas.

La consulta combina LIKE y el operador <% de similitud de palabra para términos de al menos cuatro caracteres. El cálculo utiliza word_similarity y pondera coincidencias por campo: título, autor y tema; añade bonificaciones por frase exacta y ordena los resultados antes de paginarlos. Se trata de búsqueda tolerante a ciertas variaciones, no de corrección universal de palabras [13].

El endpoint de listado GET /api/v1/libros proporciona filtros distintos del endpoint /buscar. Por ello, la búsqueda tolerante no debe describirse como si admitiera automáticamente todos esos filtros en una sola petición. Los índices mejoran las opciones de ejecución disponibles para PostgreSQL, pero cualquier afirmación cuantitativa sobre tiempos de respuesta requeriría mediciones con datos representativos.

10. Conclusiones

Consulta Fácil separa la experiencia de consulta del catálogo, la ejecución de casos de uso, las reglas del dominio y el acceso a PostgreSQL. Los niveles C4 muestran la solución desde los actores del CLENA hasta el flujo de búsqueda en NestJS. La interfaz LibroRepository y el módulo de composición permiten sustituir la tecnología de persistencia sin hacer depender directamente el caso de uso del cliente Prisma.

Prisma facilita la persistencia tipada y las migraciones; ValidationPipe concentra la validación de entrada HTTP; y Pino estructura el registro de eventos. Las extensiones de PostgreSQL complementan al ORM cuando la búsqueda requiere normalización de acentos y similitud de términos. El alcance de las vistas documenta las integraciones presentes en el proyecto, sin atribuirle servicios externos que no forman parte del despliegue descrito.

11. Referencias

[1] Corporación Luis Eduardo Nieto Arteta (CLENA), “Inicio,” *clena.org*. https://www.clena.org/

[2] Universidad del Norte, “Entrega 1: Proyecto Final Backend, Soluciones para el CLENA,” Enunciado de curso, 2026.

[3] S. Brown, “The C4 model for visualising software architecture,” *c4model.com*. https://c4model.com/

[4] R. C. Martin, *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Hoboken, NJ: Pearson, 2017.

[5] NestJS, “Custom providers,” *docs.nestjs.com*. https://docs.nestjs.com/fundamentals/custom-providers

[6] Prisma, “What is Prisma ORM?,” *Prisma ORM v6 Documentation*. https://www.prisma.io/docs/orm/v6/overview/introduction/what-is-prisma

[7] TypeORM, “Getting Started,” *typeorm.io*. https://typeorm.io/docs/getting-started/

[8] NestJS, “Validation,” *NestJS Documentation v11*. https://docs.nestjs.com/v11/techniques/validation

[9] Pino, “pino: JSON logger,” *GitHub*. https://github.com/pinojs/pino

[10] Winston, “winston: A logger for just about everything,” *GitHub*. https://github.com/winstonjs/winston

[11] J. Lobo y equipo, “Clena-catalogo-biblioteca,” *GitHub*, rama dev. https://github.com/Jlobo0210/Clena-catalogo-biblioteca/tree/dev [Consultado: 11 nov. 2026].

[12] MUI, “Material UI: Overview,” *mui.com*. https://mui.com/material-ui/getting-started/

[13] PostgreSQL Global Development Group, “pg_trgm,” *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/pgtrgm.html

[14] PostgreSQL Global Development Group, “unaccent,” *PostgreSQL 16 Documentation*. https://www.postgresql.org/docs/16/unaccent.html